SISU

The Agentic Financial Operating System | Junction 2026 | Complete Project Specification

Working name: Sisu. Product and implementation brief intended for handoff to Claude Code. This describes the proposed final product, target user, technical architecture, demonstration, datasets, sponsor integrations, constraints and submission requirements. It is a product specification, not a claim that the product has already been built.

1. Executive concept and pitch

Sisu is a mobile-first, privacy-conscious financial agent for young Finnish and European adults who struggle to translate their banking history into confident financial decisions. Existing banking interfaces explain what happened. Sisu identifies financially meaningful patterns, investigates current real-world alternatives, simulates what changes would mean for the customer's future, teaches the relevant concepts and completes the tedious parts of the next step—subject to user approval.

Core proposition: “Your bank knows your past. Sisu helps change your future.” Supporting hook: “We find the money you didn't know you were losing, and help you do something about it.” The concept is an agentic financial coach, not a stock-picking bot, passive dashboard or generic chatbot.

Primary problem: Young adults have valuable transaction histories, fragmented financial knowledge and numerous online options, but limited ability or time to join them into evidence-based, personalized action. A bank has deep context about the customer's money; the web has today's prices, rules, options and opportunities. Sisu bridges those two worlds, with visible provenance and guardrails.

Who benefits

Primary users: Finnish young adults around 18–30, including people moving out, renting, budgeting for the first time, considering a mortgage, starting an emergency fund or learning about investing. Secondary users: families and other retail-bank customers facing recurring-bill, rent, savings and mortgage decisions. Institutional stakeholder: S-Pankki and its S-mobiili digital banking ecosystem. Technology stakeholders: Exa, NVIDIA, Verda, ElevenLabs, Context.dev, and optionally Modal. The developer team and judges are demo stakeholders.

Why S-Pankki: S-Bank describes itself as Finland's fourth-largest bank with 3.5 million customers and reports that S-mobiili is used weekly by one in three Finnish adults. Its challenge explicitly prioritizes trusted conversational banking, financial literacy, personalization, three spending baskets, what-if guidance, mobile UX and a realistic path to integration. Sisu should feel like a native extension to S-mobiili, not an unrelated startup product.

2. Product: what Sisu actually does

A. Financial X-Ray — deeply personalized banking intelligence

Users choose a synthetic S-Pankki customer. Sisu analyzes transactions, income and outflows, merchant recurrence, balances over time, cash-flow seasonality, potentially avoidable expenses and unusual changes. It assigns expenses to the three required pedagogical baskets: (1) mandatory regular costs such as rent, (2) mandatory irregular costs such as repairs, and (3) everyday discretionary spending that quietly accumulates. The UI allows open-ended questions over the user's own data, such as “Where did my money go?”, “How much did I spend on taxis last month?”, “What kind of spender am I?”, “What can I safely spend this week?” and “What if I spent 20% less on dining?”
This layer must compute figures from structured queries and deterministic financial code. An LLM can interpret the question and explain results, but must not invent balances, amounts, transaction dates, merchant names, eligibility or investment returns. Show traceable transaction groups behind every material claim. Any inferred recurring bill or insight should carry confidence and allow correction.

B. Opportunity Hunter — Exa-powered internet butler

The agent turns patterns into possible actions: compare telecom or energy contracts; investigate rent alternatives and applicable rules; examine public support/benefit opportunities; compare bank accounts, savings products or mortgage terms; research an apartment; inspect ETF factsheets and fee structures. Exa must retrieve live facts that model training alone cannot know, with URLs, access dates, quoted facts or extracted fields, caveats and source quality. The result is not “Here are three links.” It is a completed research task: screened alternatives, comparative calculations, relevant eligibility or switching requirements, and a prepared next step.
Agent workflow: recognize opportunity → establish current baseline from bank data → formulate targeted web queries with minimum necessary information → search Exa → optionally parse candidate pages/PDFs using Context.dev → cross-check current offers and terms → calculate net benefit and confidence → explain trade-offs → draft a message, negotiation request, application checklist or simulated instruction → request explicit user approval → execute only in a safe connected or mocked environment → record outcome and source provenance.

C. Financial Futures — personalized what-if simulations

A distinctive interactive “Financial Multiverse” compares baseline and alternative futures. Start with moving out: “Can I afford an apartment in Helsinki while still saving and investing?” Sisu combines cash-flow history, live rental research, reasonable assumptions about housing costs, an emergency buffer and the impact on investable surplus. Other scenarios: a 15% rent increase, job loss for three months, cutting discretionary spending, renegotiating a bill or mortgage, making a monthly savings contribution, or choosing between debt repayment and investing.
Visual presentation: side-by-side branching futures, live slider adjustments, projected cash flow, emergency runway, safe-to-spend and safe-to-save estimates, total costs, opportunity

cost and future wealth. Treat return assumptions as scenarios, not predictions. Show uncertainty, potential investment loss, fees, taxes and inflation where relevant. User changes one input and sees both the calculation and the coach's explanation update immediately.

D. Investment and wealth education

Extend the coach beyond budgeting into emergency funds, the cost of debt, cash versus investments, diversification, index funds and ETF composition, fees, inflation, investment time horizons and risk tolerance. The user can ask what happens if €125 per month previously spent on dining is saved or invested. Use sourced factsheets through Exa and verified formula-based projections, not personalized speculative stock picks. Investment content is educational, with clear assumptions and no autonomous real-money trading in the hackathon MVP.

E. Embedded financial literacy

Financial literacy must be part of each decision, not a detached course tab. Sisu teaches at the moment of relevance. It explains why expenses fall in each of the three S-Pankki baskets; asks “Predict before you reveal” microquestions before displaying a simulation; offers quick explanations of compounding, liquidity, emergency funds, opportunity cost, taxes, risk and mortgage affordability; and remembers which concepts the user has already understood. Judge-visible learning: show an initial misconception, user prediction, evidence-based outcome and short tailored explanation. Measure progress through scenario decisions and short comprehension checks, not arbitrary engagement metrics.

F. Persistent personalization and proactive work

With consent, remember customer goals, past conversations, preferences, corrections and prior approvals, such as wanting an apartment near Helsinki metro, a 30-minute commute ceiling or a planned emergency fund. Memory is stored in a controlled backend database and retrieved when relevant, not automatically baked into model weights. In a more mature deployment the agent can monitor offers, policy changes or recurring financial issues and notify the user. For Junction, demonstrate a persisted preference across sessions or an on-demand fresh research task rather than claiming continuously autonomous monitoring that does not exist.

3. Flagship 90-second live booth experience

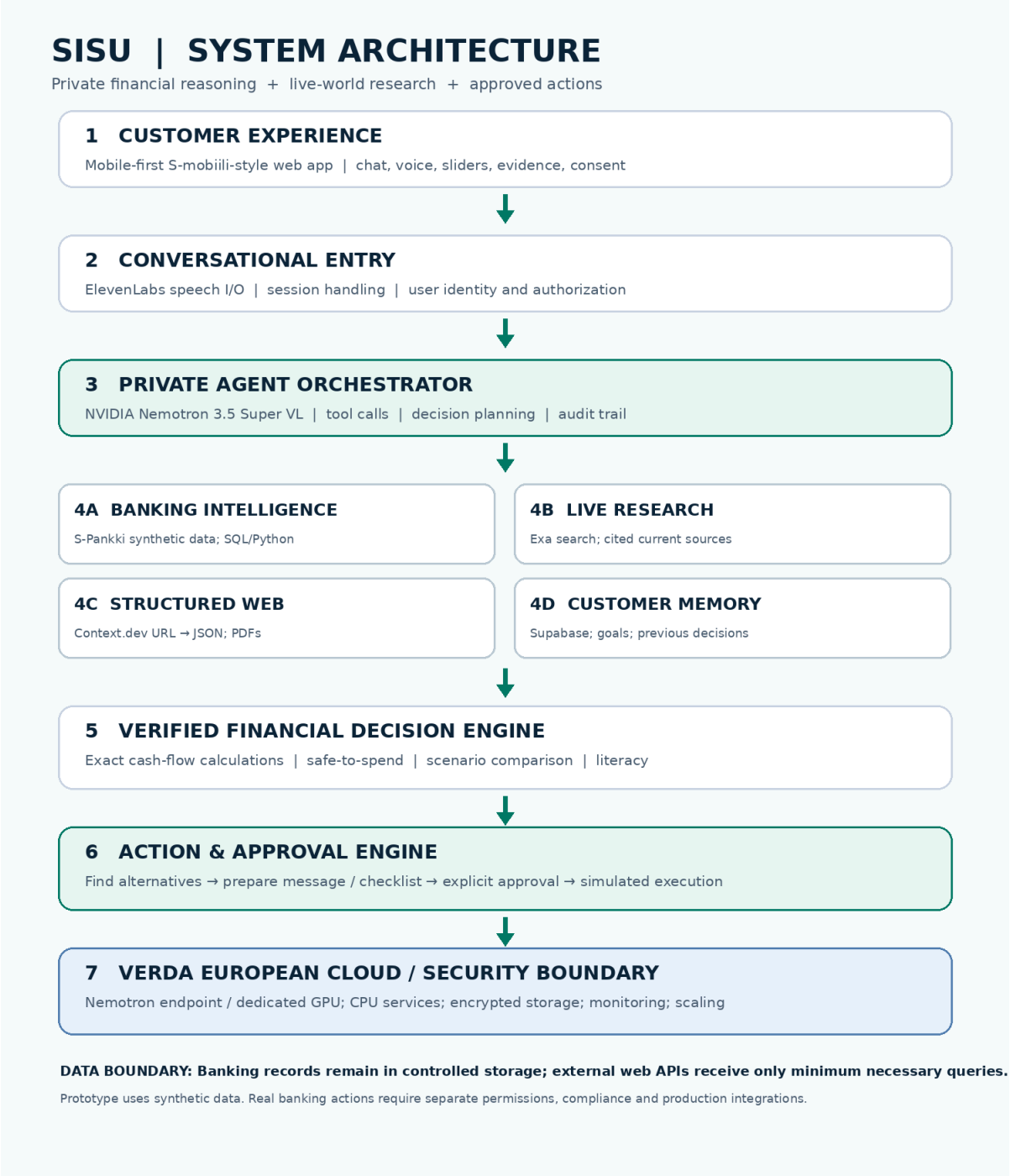
1 | Select an attractive synthetic S-Pankki customer profile on a mobile-first interface. Show animated transaction analysis, spending baskets and 2–3 verifiable opportunities. A reviewer can inspect the specific records and see exactly why the agent flagged an expense.
2 | Reviewer asks by voice: “Can you find a cheaper option for this recurring bill?” Alternatively: “Can I afford to move to Helsinki?” or “What would happen if I lost my job for three months?” ElevenLabs transcribes and speaks; Nemotron plans tool calls; Python/SQL computes baseline.
3 | Live Exa search retrieves up-to-date alternatives or relevant support rules, and Context.dev extracts structured offer conditions, fees or PDF terms when needed. Project

source links and “retrieved just now” evidence beside the actual comparisons. Judges should be able to vary the query; never rely on screenshots of canned results.
4 | The Financial Multiverse dramatically branches into “do nothing”, “switch/save”, and “save/invest” alternatives, with precise math and clear assumptions. Voice questions or sliders change figures in real time. Insert a teaching interaction: “Which future is safer if you suddenly lose income?”
5 | The agent prepares a negotiation email, application checklist or safe simulated recurring-savings instruction. Reviewer approves; the simulated action is completed and the financial future recalculates. The exact on-screen payoff: personal-data insight → live factual research → understandable recommendation → prepared/approved action → measurable simulated impact.
Booth design requirements: a striking, polished mobile UI; readable large-screen mirror; speed; judge-friendly example profiles; one-click resets; strong loading and failure states; evidence drawer; voice optional with text fallback; robust handling of arbitrary judge queries. Avoid faked live calls. Demonstrate an actual endpoint request and the output it produced.

4. Layered technical architecture

The architecture is deliberately modular. The secure customer data and financial calculations are independent of the conversational language model. External web tools see only the smallest query needed; they do not receive raw transaction histories. All tool results have source and freshness metadata. Use a user-approved action boundary before any external write or simulated banking change.

Architecture diagram — the complete stack



Layer 1 — Experience: Next.js/React mobile-first application that can plausibly be embedded in S-mobiili; spending charts, interactive future projections, sources, education prompts, opportunity feed and permission/approval cards. Responsive web deployment and QR-link access for peer reviewers.

Layer 2 — Conversation: ElevenLabs voice input/output with low-latency turn-taking; text remains the primary accessible fallback. ElevenLabs connects to the application orchestration endpoint, not directly to raw bank tables. Model voice/agent features should be tested for latency and reliability.

Layer 3 — Reasoning and orchestration: NVIDIA Nemotron 3.5 Super VL as the required core reasoning model for the NVIDIA track. It interprets intents, chooses tools, handles grounded research, reads supporting PDFs/screenshots where useful, structures answers and initiates controlled task plans. Keep an inspectable trace of its tool calls and why each was made. NVIDIA NeMo is a training/customization framework, not itself the inference model; optional offline evaluation or tuning is a later extension, not a hard dependency.
Layer 4A — Banking intelligence: S-Pankki synthetic transactions and customer data in DuckDB/PostgreSQL/Supabase, queried via constrained APIs and deterministic SQL/Python calculations. Mortgage event data lives in a separate joined data model. Implement secure authorization and per-customer filtering before the model receives any results.
Layer 4B — Live retrieval: Exa Search API (fast/balanced/deep) finds recent offers, laws, relevant public resources, factsheets and comparable listings; Contents retrieves readable pages and source excerpts. Websets and Monitors may support future verified discovery and recurring checks. Preserve links, retrieval time and evidence.
Layer 4C — Structured web: Context.dev is NOT conversational memory. It extracts selected live pages or documents into typed JSON or Markdown, e.g. provider, monthly price, introductory-period duration, setup fees, cancellation conditions, eligibility criteria and effective date. An endpoint call must be core to the live demo for its prize, and visibly drive the actual comparison. Check success flags, freshness (maxAgeMs: 0 when necessary) and extraction validation.
Layer 4D — Memory: Supabase/Postgres + vector retrieval where useful; session state, consented personal preferences, goals, explanations and decisions. Separate short-term context from durable memory, encrypt and allow deletion/correction. The model does not need to retrain itself after each interaction.
Layer 5 — Financial simulation: audited Python/TypeScript computations for cash-flow trends, safe-to-spend, risk buffers, expected expenses, savings trajectories, debt trade-offs, investment projections and scenario sensitivities. Do not let Nemotron perform critical arithmetic without tool verification. Store assumptions and provenance with each result.
Layer 6 — Action workflow: State-machine-driven agent for research → recommendation → draft → approval → safe execution. Working demo examples: draft a landlord/bill negotiation email, create a benefit application checklist, generate a provider-switch request, or create a savings rule inside a mocked S-mobiili environment. Confirm before sending, changing a record or initiating any financial instruction. Every action has a visible status and audit log.
Layer 7 — Compute / hosting: Verda European cloud to host Nemotron via its NVIDIA-associated endpoint or on a suitable GPU deployment, plus agent backend on CPU instances and other services. Verda is not mandatory for product functionality, but is a strategic match for keeping inference under European operational control. If custom model weights/fine-tuning are required, use appropriately provisioned GPU instances/containers rather than assuming a managed endpoint supports arbitrary fine-tunes. A secure bank deployment additionally needs due diligence, regulated outsourcing controls, encryption, isolation and operational resilience.
Alternative infrastructure: Modal could serve the model, tool execution or isolated agent code, but competing simultaneously on Verda and Modal can dilute the clear European infrastructure narrative. Recommend Verda as primary; use Modal only for a distinct compute-intensive component that materially improves the demo. Avoid a fragmented architecture deployed across many clouds solely to enter prizes.

5. Dataset specification and handling

Official synthetic dataset repository: https://github.com/hackjunction/Junction-2026-S-pankki . The provided S-Pankki PDF brief is five pages. Page 2 summarizes the mobile-first coach, user questions, three spending baskets and MyData principles; page 3 specifies ten transaction fields; page 4 specifies customer fields; page 5 specifies mortgage event fields.

Transaction data — tx_junction_transactions.csv

The transaction file is about 346 MB, Git LFS managed, with columns: customer_id; bank_account_id; tx_date; tx_timestamp; category; tx_amount_eur (negative for outflows and positive for inflows); balance_after_eur; transaction_type; merchant_id; mcc_code. Missing merchant/MCC can be encoded as '.' or blank for non-card transfers. Preserve dates and signs correctly. Verify download size: a roughly 134-byte pointer that begins “version https://git-lfs.github.com/spec/v1” is NOT the CSV. Use Git LFS or the repository's Download Raw option.

Customer data — tx_customers_junction.csv

Contains customer_id, customer_postal_code and customer_age. Postal code MUST be treated as text to retain leading zeroes (e.g. 00990, 00530). The combination supports segment views and regional benchmarks, but age or postcode alone must not be treated as proof of benefit eligibility, personal circumstances or risk tolerance.

Mortgage datasets

mtg_negotiation_history.csv includes event-level mortgage application data with 16 named fields: mortgage_customer_id; timestamp; event_type; property_id; property_type; property_price_estimate; loan_amount; equity_amount; collateral_type; collateral_value; collateral_status; decision; rejection_reason; expense_type; expense_amount; note. The history captures events such as expense_recorded, collateral_added and collateral_valuation, as well as possible final approvals/rejections and reasons. mtg_tx_customer_link_and_final_mtg_infio.csv joins mortgage_customer_id to transaction customer_id and provides final application outcome information. Treat historical application status, requested loan and collateral value separately from current outstanding mortgage balance, interest rate and monthly payment; the brief does not establish that those present-day parameters exist for every user. Ask for missing values or label demo assumptions clearly. This mortgage view supports a possible flagship Negotiation/Readiness Agent and deeper bank-specific personalization.
Initial pipelines: load synthetic CSVs with UTF-8 BOM-aware encoding (utf-8-sig); normalize customer/account keys and timestamps; index by customer/date; precompute recurrence and rolling cash-flow features; redact or minimize fields before inference; expose read-only financial-query functions. Do not assume all external offers or investments are linked in the bank dataset.

6. Three concrete agent workflows

Workflow A — Recurring Expense Negotiator (primary end-to-end execution demo)

The bank data shows a recurring telecom expense. Sisu identifies the charge and frequency, asks whether the customer wants alternatives, uses Exa to find current suitable Finnish plans, uses Context.dev structured extraction to retrieve monthly price, contract term, restrictions, setup/switching fees and eligibility, computes net annual savings, cites each source, drafts a provider inquiry or cancellation/switching checklist, requests approval, and executes a safe simulated action. Then the projection updates. Avoid claiming merchant identity where dataset only has a synthetic merchant ID; map to a clear fictional illustrative provider or collect the actual provider from the user.

Workflow B — First Apartment / Benefits Butler

The user wants to move out in Helsinki. Sisu uses spending and cash-flow data, Exa's live listings and relevant public guidance (including Kela or other official sources), optionally Context.dev extraction, and a what-if engine to show feasible rents and trade-offs. It finds qualifying-looking support possibilities but does not state final eligibility without necessary verified facts. It can create a research-backed apartment shortlist, inquiry drafts and an application checklist; user approves all outbound actions.

Workflow C — Mortgage Readiness / Negotiation

For customers with mortgage application events, join historical mortgage steps with cash-flow and income signals; identify application status and known constraints; compare current mortgage information or offers sourced through Exa; display terms, fees, affordability and future payments; prepare lender questions or a negotiation brief. Do not imply that historical mortgage application data reveals current loan APR or that another bank will approve a customer. Mortgage product suitability and eligibility remain subject to lender assessment.

7. Trust, privacy, European AI and model customization

A key product argument is that S-Pankki's underused financial data is the defensible intelligence. Nemotron is an open-weight model family that can run on controlled infrastructure, and Verda operates European cloud services. That enables a potentially more controlled data-processing and deployment architecture than relying exclusively on third-party hosted proprietary inference. HOWEVER, third-party APIs do not inherently transfer ownership of data or necessarily violate bank confidentiality; privacy depends on contracts, processing terms and technical safeguards. Treat sovereign/private inference as a design objective, not an unverified compliance guarantee.
Data minimization: raw transactions stay in controlled storage; the model is given customer-authorized aggregates or selected facts, not unlimited unrestricted table access. Exa/Context.dev queries should not contain personal transaction histories, identifiers or

private financial circumstances except carefully approved minimal context. Use encrypted storage and TLS, least-privilege tool scopes, logs, per-customer authorization, explicit consent and data retention rules. Separate the synthetic hackathon sandbox from any prospective production environment.
NeMo training is optional. Continuous on-the-fly fine-tuning on customers' private conversations is NOT the intended hackathon architecture. Personalization should primarily come from secure retrieval and consented persistent memory. Future bank-specific fine-tuning could train on permissioned, appropriately processed proprietary data using evaluation pipelines, subject to security, licensing and model risk management. NVIDIA judges require actual Nemotron 3.5 Super VL use; merely mentioning NeMo is insufficient.
Safe execution: financial advice is educational, calculations are auditable, sources are linked, uncertainty is displayed, and potentially consequential steps require customer approval. No unauthorized emails, forms, bank transfers, account modifications or trades. Mock execution must be clearly labeled. A complete action is a verified state change or prepared/sendable artifact—not a mere conversational promise.

8. Challenge alignment and judging requirements

S-Pankki — Personal Economic Trainer (primary product track)

Judging: financial literacy/pedagogical impact 20%; originality of AI integration 20%; clarity of mobile UX 20%; feasibility 20%; potential S-mobiili integration 20%. First prize €2,000 Airbnb gift card; second €1,000; third €500. Requirements implied by brief: large synthetic dataset, conversational financial data access, mandatory regular / mandatory irregular / everyday spending basket explanation, what-if and safe-to-spend/save guidance, open-ended MyData exploration and mobile-app-compatible design. Statistics Finland/open benchmarks and PSD2 multi-bank access are optional extensions.

Exa — Internet Butler / Your Personal Assistant (core action track)

Judging: meaningful use of Exa 20%; usefulness to a specific group 20%; clarity of sources 20%; originality 20%; working implementation on live judge queries 20%. First $2,500 cash + $5,000 Exa credits; second $1,500 + $3,000 credits; third $500 + $1,000 credits, plus merch. Exa must discover current non-memorized information, not act as a decorative citations button. Search and Contents APIs are primary; filters, structured outputs, Websets, Monitors, hosted MCP and SDKs are available. $50 starter credit code: EXA50JUNCTION; redeem at https://dashboard.exa.ai/billing . Examples: https://demos.exa.ai/ .

NVIDIA — Best Use of Nemotron (central reasoning model)

Minimum requirement: NVIDIA Nemotron 3.5 Super VL must be used in the working solution; clearly show the model's core contribution; submit a short explanation of exact model, role, importance and quality improvements. This 120B-parameter/12B-active-per-token vision-language model is described in the challenge as multimodal and capable of tool-driven workflows. Judging: meaningful usage 25%,

technical execution 25%, output quality 25%, creativity/differentiation 25%. Prize: two winning teams each receive an RTX 5080. NVIDIA Build API documentation lists base URL https://integrate.api.nvidia.com/v1 and examples such as nvidia/nemotron-3.5-lightning-30b-a3b; that example model ID is not a substitute for the expressly required Super VL. Verify the actual Super VL endpoint/weights with NVIDIA and Verda mentors. Contact: ewinch@nvidia.com .

Verda — Best Use of Verda (preferred deployment track)

Judging: ambition 25%; right Verda product fit 25%; working prototype/documented process 25%; credible path to scale 25%. Verda explicitly offers keeping banking-coach inference in Europe on open-weight models as an example. Relevant services: hosted NVIDIA Nemotron endpoint, Inference API, GPU instances / Instant Clusters, CPU instances, auto-scaling containers and batch jobs. It hosts European data centres and states 100% renewable energy use. Document what compute enabled that a laptop couldn't; share a summary link in Verda's Discord. Friday 20:30 workshop, third floor Booth 6. First prize $10,000 cloud credits; second/third $2,500 credits. Credits information: https://verda.com/blog/junction-2026 (the supplied challenge text contained a malformed hackjunction-wrapped link; use the canonical address after confirming access).

Context.dev — Best Use of Fully Structured Web (valuable secondary track)

Minimum: call at least one real Context.dev API endpoint in the CORE user flow, visibly drive the demo with its data, submit GitHub repo + demo + endpoints used/why. Judging: creative use 40%, centrality of live web data 30%, technical execution 30%. Recommended use: extract current contract terms or mortgage product fee details from Exa-discovered pages into JSON, then visibly drive savings comparison. It is a web-data API, not persistent conversation memory. API supports scrape, extract, news, brand data, file parsing, monitors, and batch. Use maxAgeMs:0 for freshness when necessary; check success flags even on HTTP 200. 50,000 non-expiring credits per team code JUNCTION26 at https://context.dev/dashboard/redeem-credits ; docs https://docs.context.dev/quickstart and https://docs.context.dev/agent-quickstart ; MCP https://mcp.context.dev/mcp ; contact beer@context.dev . Prize €750 first, €250 second, swag third.

ElevenLabs via MLH — Best Use of ElevenLabs

Use ElevenLabs for natural bidirectional voice interaction, not merely a canned speech clip. Spoken questions should reach the real orchestration endpoint and the agent should answer with personalized calculations. Prize: wireless earbuds. Get resources at https://mlh.link/junction-26 and https://mlh.link/elevenlabs . Make sure a noisy booth does not make voice the only way to use Sisu.

Modal — alternate / optional compute prize

Modal is a potential alternative to Verda for GPU serving, isolated agent sandboxes, autoscaling web endpoints and training. Its judging weights: effective use 40%, ambition

25%, impact 20%, demo 15%; prize $2,000 cash and $3,000 credits. $100 credits per participant via https://modal.fillout.com/t/qMXCmRGseUus?code=VVX-2QX-8HW . Do not integrate Modal purely for an extra logo; use it only if one component clearly requires its isolated sandbox, fast scaling or other distinct service. Verda is strategically preferred for the European banking/cloud narrative.

9. Implementation expectations for Claude Code

Build a production-shaped hackathon demo, not a slide prototype. Use modular front-end, typed API contracts and tool adapters. Suggested stack: Next.js + React + TypeScript; Tailwind / accessible component library; Python FastAPI or TypeScript backend; DuckDB for fast local analytical queries; Supabase/Postgres for customer sessions/memory; NVIDIA Nemotron reasoning through Verda (verify actual Super VL) with a fallback path for testing; Exa SDK for search/contents; Context.dev for structured extraction; ElevenLabs for voice. Host a public mobile-friendly URL, with no credentials leaked to the client.
Recommended services/modules: banking_ingest; customer_profile; basket_classifier; financial_tools; safe_to_spend; scenario_engine; opportunity_detector; exa_research; context_extractor; evidence_validator; nemotron_orchestrator; elevenlabs_voice; memory_service; action_workflows; approval_gate; demo_reset; audit_events. Provide clear interfaces and allow any external provider to be stubbed for local tests while retaining genuinely live calls in judging mode.
Acceptance tests: (1) transactions reconcile by customer and date; (2) three basket totals sum to their defined expense universe; (3) source-linked offers are current and fee-adjusted; (4) Context.dev extracted JSON visibly changes a calculation; (5) Nemotron actually chooses and executes tools; (6) investment scenarios report their assumptions and do not guarantee returns; (7) no action executes without an approval event; (8) a restarted session can retrieve approved user preferences; (9) site is usable on phone; (10) arbitrary live judge query gets a source-backed result or an honest, useful limitation.
Demo reliability: keep 2–3 known-good synthetic profiles, deterministic fixture tests, graceful fallbacks, server-side API keys, request timeouts, cost/latency budgets and logs. Display live-query loading states, avoid fabricated status labels, prefetch only clearly marked background supporting data. Record a backup video, but the default demonstration should be live and judge-interactive.

10. Scope and product differentiation

The central innovative combination is: underused transaction intelligence + open-weight bank-controlled reasoning + live web investigation + structured extraction + financial simulations + contextual teaching + concrete agent action. No single component is asserted as globally unprecedented. The memorable innovation lies in integrating them into a verified, personalized end-to-end workflow that works with real prompts.
Primary working experience: recurring expense discovery and agentic switching/negotiation in a safe mock environment. Secondary working experience: “Can I afford to move out?” with live rental research and financial futures. Tertiary optional experience: mortgage readiness and investment education. It is better to have a compelling complete workflow and two robust adjacent queries than many incomplete features.

Explicit non-goals for the hackathon: autonomous live stock trading; guaranteed financial returns; unsolicited sending of private data to web APIs; continuous uncontrolled fine-tuning on private customer records; unverified legal/benefit eligibility judgments; direct live S-Pankki integrations without credentials/authorization; claims that simulated actions are real bank transfers or successful provider switches.

11. Source links and sponsor resources

Official challenges landing page: https://hackjunction.app/participate/45/challenges
S-Pankki synthetic data and README:
https://github.com/hackjunction/Junction-2026-S-pankki
S-Pankki institution: https://www.s-pankki.fi/
Exa demos: https://demos.exa.ai/ | Credits: https://dashboard.exa.ai/billing | Company:
https://exa.ai/
Verda: https://verda.com/ | Junction offer: https://verda.com/blog/junction-2026
NVIDIA Build endpoint: https://integrate.api.nvidia.com/v1 | NVIDIA:
https://www.nvidia.com/en-us/
Context.dev: https://context.dev/signup | Docs: https://docs.context.dev/quickstart | Agent guide: https://docs.context.dev/agent-quickstart | MCP: https://mcp.context.dev/mcp | Redeem: https://context.dev/dashboard/redeem-credits
ElevenLabs via MLH: https://mlh.link/junction-26 | https://mlh.link/elevenlabs
Modal: https://modal.com/ | Credits:
https://modal.fillout.com/t/qMXCmRGseUus?code=VVX-2QX-8HW
Attached source brief: s-pankki_junction_2026_brief.pdf (five pages; the document's dataset fields and three spending categories have been integrated above).

12. Handoff instruction

Claude Code should implement Sisu as an inspectable, mobile-first working system. Begin by validating the S-Pankki data load and real sponsor API availability, particularly the exact Nemotron 3.5 Super VL endpoint and Verda deployment capability. Preserve the architecture's data boundary, show sources and assumptions, implement approval-gated simulated actions and optimize for a stunning but truthful peer-reviewer demo. Do not add integrations that merely increase sponsor count without improving the core user journey.

13. Three-person delivery team and Git workflow

Team split: one owner for Product/UI, one for Financial Data & Deterministic Backend, and one for AI Agent & Integrations. All three work in the same GitHub repository using separate feature branches and agreed API interfaces. Everyone is responsible for their own tests and short demo handoff; the team jointly owns the final pitch and integration.

TEAM MEMBER A — Frontend / Product Experience (branch feature/frontend)

Owns mobile-first Next.js UI, visual design system, onboarding/profile selector, three-basket spending X-Ray, Opportunity Feed, source evidence drawer, dynamic Financial Multiverse, voice controls, approval cards, responsive design and booth-demo experience. Develop first against typed mock API responses; replace mocks as backend endpoints become available. Acceptance: polished mobile experience, no hardcoded results masquerading as live, all main states covered (loading/error/approval), visual impact on large screen.

TEAM MEMBER B — Banking Data / Financial Engine (branch feature/financial-engine)

Owns Git LFS dataset ingestion, customer and mortgage data joins, normalized schema, read-only account APIs, transaction grouping and recurrence, three baskets, safe-to-spend, deterministic savings/mortgage/investing projections, scenario endpoint, validation and test fixtures. Builds synthetic demo profiles with defensible calculations. Acceptance: reconciled totals, fast customer queries, clearly marked assumptions, no invented mortgage balances or merchant identities.

TEAM MEMBER C — Agent / AI / Sponsor APIs / Cloud (branch feature/agent-orchestration)

Owns Nemotron 3.5 Super VL call path and tool orchestration, Exa live search and Contents, Context.dev JSON extraction, ElevenLabs voice bridge, memory integration, citations/provenance, opportunity-research workflow, approval-gated action state machine, Verda deployment, logs/retries/latency and sponsor requirements. Acceptance: real demonstrated Nemotron tool use, live Exa retrieval, Context.dev data visibly changing calculations, safe action handoff, stable public backend endpoint.

Shared API contract — agreed BEFORE independent coding

Agree shared TypeScript/Pydantic schemas in /packages/contracts or /shared. Minimum endpoints: GET /api/customers/demo; GET /api/customers/:id/overview; GET /api/customers/:id/transactions?from&to; GET /api/customers/:id/opportunities; POST /api/scenarios; POST /api/agent/query; GET /api/research/:runId; POST /api/actions/prepare; POST /api/actions/:id/approve; POST /api/demo/reset. Responses should include customerId, dataTimestamp, currency, assumptions, citations, confidence and actionStatus where relevant. Never expose API keys or bank tables directly to the browser.

Recommended repo shape

/apps/web — Next.js UI (A)
/apps/api — financial data APIs and orchestration endpoints (B + C in separate modules)
/packages/contracts — shared schemas and generated types (joint; one designated maintainer)

/packages/financial-core — deterministic calculators and tests (B)
/packages/agent — Nemotron/Exa/Context/voice/action tools (C)
/data — synthetic fixtures and ingestion scripts (B); large/raw customer data excluded from git commits unless appropriate Git LFS policy
/docs — README, architecture, API examples, sponsor-specific evidence and demo script (joint)

Parallel development and Git rules

1. Create main protected branch, and feature/frontend, feature/financial-engine, feature/agent-orchestration branches. Commit initial app skeleton and shared contracts to main first.
2. Use pull requests (PRs) instead of directly pushing work to main. Each PR needs a short summary, successful build/tests, example request/response for changed endpoints, and a reviewer from another role. Small PRs throughout the hackathon beat one giant final merge.
3. Both backend owners work under separate directories/modules to minimize collisions. UI owner depends only on shared contracts, not another teammate's temporary implementation. Use mock adapters until integrations are live.
4. One person acts as integration lead (recommend the backend/data owner, because calculations and API correctness are core). Integration lead merges after checks, coordinates environment variables and ensures frontend/backend contract stability; do not make this person rewrite teammates' features.
5. Continuous integration: run install, lint, typecheck, unit tests and a smoke test for the public demo. Require a reproducible .env.example without secret values. Never commit keys. Use staging deploy previews and a known-good main branch.
6. Integration milestones: first, transaction data + UI stub + agent stub all obey the shared contract; next, complete one real end-to-end expense opportunity with live Exa/Context and calculation; then, connect ElevenLabs and Verda; finally, test peer-reviewer prompts, rehearse and freeze optional additions. Plan parallel work, not sequential handoffs.
7. Conflict prevention: Git worktrees are optional for multiple agents working from one machine, but each teammate's separate clone and branch is sufficient. Rebase or merge main regularly, communicate schema changes immediately and never silently rename endpoint fields. Keep a rollback tag for the last working demo.
8. Scope discipline: if short on time, prioritize the complete live Expense Negotiator demo, then the move-out scenario, and only then add additional investment/mortgage or sponsor embellishments. Assign one designated speaker and a second person as live-demo operator while the third handles judge questions and failure recovery.

Cross-functional demo definition of done

A peer reviewer can open the deployed URL, choose a real synthetic customer, see correctly computed spending insights, ask a new question by text or voice, watch Nemotron invoke tools, inspect Exa and Context.dev sources, change a scenario, understand one financial lesson, approve a prepared simulated action and see the financial impact update. Everyone can explain their component and the shared flow in 30 seconds.